

Software Requirement Specification

ON

CampusTwin - AI DRIVEN SMART CAMPUS DIGITAL TWIN

Master of Computer Applications (MCA)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY
PHAGWARA, PUNJAB



From 02/12/26 to 05/12/26

Project Guide

Dr. Tarandeep Singh Walia
Associate Professor
School of Computer Applications

SUBMITTED BY:

Name of Student:	Registration Number:	Roll Number:
Pritam Kumar Branwal	12503898	51
Aryan Guleria	12522004	37
Dilip Kumar	12520416	58
Abhishek Mishra	12526538	55

To whom so ever it may concern

We, the undersigned students, hereby declare that the project titled **“CampusTwin - AI DRIVEN SMART CAMPUS DIGITAL TWIN”**, carried out by us under the guidance of **Dr. Tarandeep Singh Walia (ID: 25153), Project Guide**, is an original work completed during the Spring Term of the academic session **2025–2027** at **Lovely Professional University, Phagwara, Punjab**.

This project report is submitted in partial fulfilment of the requirements for the award of the degree of **Master of Computer Application (MCA)**.

We further declare that this work has not been submitted previously, either in part or in full, for the award of any other degree or diploma at any institution.

Pritam Kumar Branwal (12503898)
Aryan Guleria (12522004)
Dilip Kumar (12520416)
Abhishek Mishra (12526538)

Dated: 28th March 2026

Table of Contents

1. Introduction	
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 References	5
2. System Overview	
2.1 Product Description	5
2.2 User Types	5
3. System Features (Functional Requirements)	
3.1 User Authentication & Authorization	6
3.2 Incident Management System	6
3.3 Smart Garbage Monitoring System	7
3.4 Alert & Notification System	7
3.5 Map-Based Visualization System	8
4. External Interface Requirements	
4.1 User Interface (UI)	8
4.2 Hardware Interface	8
4.3 Software Interface	9
5. Non-Functional Requirements	
5.1 Performance Requirements	9
5.2 Security Requirements	9
5.3 Scalability Requirements	10
5.4 Reliability Requirements	10
5.5 Usability Requirements	11
5.6 Maintainability Requirements	11
6. System Design Approach	
6.1 Architecture Overview (Microservices Architecture)	11
6.2 Module Description	12
6.3 Data Flow Overview (DFD – 0 & DFD – 1)	12
6.4 Design Principles Used	13
6.5 System Architecture and ER Diagram	14
7. Technology Stack	
7.1 Backend Technologies	15
7.2 Frontend Technologies	16
7.3 Database	16
7.4 Messaging & Communication	17
7.5 Tools & Frameworks	17
8. Future Scope	18
9. Appendix	18

1. INTRODUCTION

1.1 Purpose

The purpose of this document is to describe the requirements and overall structure of the **CampusTwin – AI-Driven Smart Campus Digital Twin** system. This document provides a clear understanding of the system's functionality, design approach, and expected behaviour.

It serves as a reference for developers, project evaluators, and stakeholders to understand how the system will be developed and how different components will interact with each other. The document focuses on defining both functional and non-functional requirements to ensure that the system meets the needs of a modern university campus.

1.2 Scope

The **CampusTwin – AI-Driven Smart Campus Digital Twin** system is designed to improve the management and monitoring of activities within a university campus. It provides a real-time digital view of campus operations to enhance safety, coordination, and overall efficiency.

The system allows users to report incidents such as medical emergencies, security issues, and maintenance problems. It also includes features like smart garbage monitoring and map-based visualization to display campus activities in an interactive way.

User access is managed through authentication and role-based authorization, ensuring that students and staff can only access relevant features. OTP-based verification during registration ensures that only valid university users can create accounts.

The system is built using a microservices architecture, making it scalable and easy to extend. Although it is designed for a university campus, it can be adapted for larger systems such as smart city applications in the future.

1.3 Definitions, Acronyms, and Abbreviations

- **Digital Twin:** A virtual representation of the physical campus that reflects real-time data and activities.
- **Microservices Architecture:** A design approach where the system is divided into smaller, independent services that communicate with each other.
- **JWT (JSON Web Token):** A secure token used for authentication and authorization of users.
- **API (Application Programming Interface):** A set of rules that allows different parts of the system to communicate with each other.
- **OTP (One-Time Password):** A temporary verification code sent to the user during registration to confirm identity.
- **AI (Artificial Intelligence):** Technology used to analyze data and assist in decision-making, such as predicting risks or prioritizing incidents.
- **SSE (Server-Sent Events):** A technique used to send real-time updates from the server to the client.

1.4 References

- Standard concepts and practices of microservices architecture and **RESTful API** design.
- Official documentation of **Spring Boot** used for backend development.
- Official documentation of **PostgreSQL** used for database management.
- **Clean Architecture** – for understanding software design principles and modular system development.
- **Designing Data-Intensive Applications** – for concepts related to scalable and reliable system design.
- **Martin Fowler - Microservices** – for understanding microservices architecture patterns.
- **Baeldung** – for Spring Boot, security, and backend development tutorials.
- **GeeksforGeeks** – for general programming concepts and system design references.
- Concepts of Digital Twin systems inspired by modern smart campus and smart city solutions.

2. System Overview

2.1 Product Description

The **CampusTwin – AI-Driven Smart Campus Digital Twin** is a web-based system designed to provide a smart and efficient way to monitor and manage activities within a university campus. It creates a virtual representation of the campus where different events and conditions can be tracked in real time.

The system allows users, especially students, to report incidents such as medical emergencies, security issues, and maintenance problems. These reports are then processed and made available to the concerned authorities like security staff or maintenance teams for quick action. In addition to this, the system also monitors the status of garbage bins and displays whether they are full or empty, helping in maintaining campus cleanliness.

One of the key features of the system is its map-based visualization, where all incidents, garbage bins, and other important data are displayed on a campus map. This provides a clear and intuitive way for administrators to understand what is happening across different areas of the campus.

The system is built using a microservices architecture, where different functionalities are handled by separate services such as authentication, incident management, and monitoring. This approach makes the system scalable, flexible, and easier to maintain.

Overall, CampusTwin aims to improve safety, efficiency, and decision-making within the campus by providing real-time information and a centralized platform for managing various activities.

2.2 User Types

The **CampusTwin** system is designed to be used by different types of users within the university, each having specific roles and responsibilities. The system provides role-based access to ensure that users can only access the features relevant to them.

- **Students:**
Students are the primary users of the system. They can report incidents such as medical emergencies, security issues, or maintenance problems. They can also view the campus map to check ongoing incidents and the status of different areas.

- **Security Staff:**
Security personnel are responsible for monitoring reported incidents related to safety and security. They receive alerts and take appropriate action based on the information provided through the system.
- **Medical Staff:**
Medical staff handle emergency situations related to health. They receive notifications about medical incidents and respond accordingly to provide assistance.
- **Maintenance Staff:**
Maintenance teams are responsible for handling infrastructure-related issues such as garbage management and repairs. They monitor garbage bin status and resolve reported maintenance problems.
- **Administrator:**
The administrator manages the overall system. They are responsible for creating staff accounts, managing user roles, and monitoring system activities. They ensure that the system runs smoothly and efficiently.

3. System Features (Functional Requirements)

3.1 User Authentication & Authorization

The system provides a secure authentication and authorization mechanism to ensure that only valid users can access the platform. It supports user registration, login, and role-based access control for different types of users.

During registration, users are required to provide their details such as name, email, phone number, and university ID. The system verifies that the email belongs to the university domain (e.g., @lpu.in) and sends a One-Time Password (OTP) to the email for verification. Only after successful OTP verification is the user account created in the system.

For login, users authenticate using their registered email and password. The system uses secure password encryption and generates a JSON Web Token (JWT) upon successful login. This token is used to authenticate the user in subsequent requests.

The system also implements role-based authorization, where access to different features is controlled based on user roles such as student, admin, security staff, medical staff, and maintenance staff. This ensures that users can only perform actions that are relevant to their responsibilities.

Overall, this module ensures secure access, prevents unauthorized usage, and maintains the integrity of the system.

3.2 Incident Management System

The Incident Management System allows users to report and track various types of issues occurring within the campus. This feature helps in ensuring quick response and better coordination among different departments.

Students can report incidents such as medical emergencies, security concerns, crowd congestion, or maintenance problems by providing basic details along with the location of the issue. Each reported incident is recorded in the system and assigned a status for tracking purposes.

Once an incident is reported, it becomes visible to the relevant authorities such as security staff, medical teams, or maintenance personnel. These users can monitor the incidents and take appropriate action based on the type and urgency of the situation.

The system is designed to handle incidents in real time and can be extended to include priority assignment and intelligent analysis in the future. This ensures that critical situations are identified quickly and handled efficiently.

Overall, this module improves response time, enhances communication between users and authorities, and contributes to a safer and more organized campus environment.

3.3 Smart Garbage Monitoring System

The Smart Garbage Monitoring System is designed to improve cleanliness and waste management within the campus. It allows the system to track the status of garbage bins placed at different locations and ensure timely maintenance.

Each garbage bin is represented digitally in the system and is associated with a specific location on the campus map. The status of the bins is updated based on predefined conditions or simulated data, indicating whether a bin is full or not. Typically, the bins are displayed in different colors, such as green for empty and red for full, making it easy to understand their condition at a glance.

When a garbage bin is marked as full, the system can notify the maintenance staff so that necessary action can be taken promptly. This helps in preventing overflow situations and maintaining a clean campus environment.

The module also provides a visual representation of all garbage bins on the map, allowing administrators and maintenance teams to monitor multiple locations efficiently.

Overall, this feature helps in optimizing waste collection processes, reducing manual checks, and ensuring better hygiene across the campus.

3.4 Alert & Notification System

The Alert and Notification System is responsible for informing relevant users about important events and incidents in real time. It ensures that the right information reaches the appropriate authorities without delay.

Whenever an incident is reported, a garbage bin becomes full, or a high crowd density is detected, the system generates alerts based on the type and priority of the situation. These alerts are then sent to the concerned users such as security staff, medical teams, or maintenance personnel.

The system follows a role-based approach, where notifications are delivered only to users responsible for handling a particular type of issue. For example, medical alerts are sent to medical staff, while maintenance-related alerts are sent to maintenance teams.

Alerts can be displayed on the dashboard and can be extended to include other notification methods such as email or mobile notifications in the future. This ensures that users stay informed and can respond quickly.

Overall, this module improves communication, reduces response time, and ensures effective coordination among different departments within the campus.

3.5 Map-Based Visualization System

The Map-Based Visualization System provides a graphical and interactive view of the entire campus, allowing users to monitor activities and conditions in real time. It acts as a central dashboard where all important information is displayed in a clear and intuitive manner.

The system displays various elements such as reported incidents, garbage bin status, and crowd density directly on the campus map. Each element is represented using markers or color indicators, making it easy to identify different types of information at a glance. For example, incidents can be shown as markers, while garbage bins can be displayed in green or red based on their status.

This visual approach helps administrators and staff quickly understand what is happening across different areas of the campus without going through complex data. It also enables faster decision-making and better coordination during critical situations.

The map updates dynamically as new data is received, ensuring that users always have access to the latest information. This feature can be further enhanced with real-time streaming and advanced visualization techniques.

Overall, the Map-Based Visualization System plays a key role in improving situational awareness and making the system more user-friendly and effective.

4. External Interface Requirements

4.1 User Interface (UI)

The user interface of the CampusTwin system is designed to be simple, intuitive, and easy to use for all types of users. It is a web-based interface that allows users to interact with the system through a browser without requiring any complex setup.

The system provides different views based on user roles. Students can access features such as reporting incidents and viewing the campus map, while staff members such as security, medical, and maintenance personnel can monitor incidents and respond accordingly. Administrators have access to management functionalities and system monitoring tools.

The interface includes forms for user registration and login, as well as input forms for reporting incidents. A key component of the UI is the interactive campus map, which visually displays incidents, garbage bin status, and crowd levels using markers and color indicators.

The design focuses on clarity and responsiveness, ensuring that users can easily navigate the system and access important information without confusion. The interface can be further enhanced to support mobile devices and additional visualization features in the future.

4.2 Hardware Interface

The CampusTwin system does not require any specialized hardware for its basic operation. It is designed to run on standard computing devices such as laptops, desktops, and smartphones with internet connectivity.

Users can access the system through a web browser, making it easily usable across different platforms. The system is compatible with common operating systems such as Windows, macOS, and Android.

In its current implementation, the system uses simulated data for features such as garbage monitoring and crowd detection. However, the design allows future integration with hardware components such as IoT sensors, CCTV cameras, or environmental monitoring devices to provide real-time data input.

This flexibility ensures that the system can evolve from a software-based solution to a more advanced smart campus system with hardware integration in the future.

4.3 Software Interface

The CampusTwin system interacts with various software components to perform its operations efficiently. The backend is developed using the Spring Boot framework, which provides RESTful APIs for communication between different modules and services.

The system uses a PostgreSQL database to store user information, incident data, and other relevant details. Communication between different parts of the system is handled through APIs, ensuring smooth data exchange and modularity.

For authentication and security, the system integrates Spring Security along with JWT (JSON Web Token) to manage user sessions and access control. This ensures that only authorized users can access protected resources.

The system is designed to support integration with additional services such as email services for OTP verification and notification systems for alerts. It can also be extended to include messaging systems like Kafka or RabbitMQ for event-driven communication in the future.

Overall, the software interfaces are designed to be modular, scalable, and easily extendable, supporting the microservices-based architecture of the system.

5. Non-Functional Requirements

5.1 Performance Requirements

The CampusTwin system is designed to provide efficient and responsive performance for all users. The system should be able to handle multiple user requests simultaneously without significant delay.

Key functionalities such as user login, incident reporting, and data retrieval should be processed quickly to ensure a smooth user experience. The system should support real-time or near real-time updates, especially for features like map visualization and alert notifications.

The backend services should be optimized to handle increasing data and user load as the system scales. The use of a microservices architecture allows different modules to operate independently, improving overall performance and reducing system bottlenecks.

Additionally, the system should minimize response time for API calls and ensure that data is processed efficiently to support timely decision-making by users and administrators.

5.2 Security Requirements

The CampusTwin system must ensure secure access and protect user data from unauthorized usage. Security is a critical aspect of the system, especially since it handles user information and sensitive incident reports.

The system implements authentication using email and password, along with OTP-based verification during registration to ensure that only valid university users can create accounts. Passwords are securely stored using encryption techniques to prevent misuse.

For authorization, the system uses role-based access control, where different users such as students, administrators, and staff members have access only to the features relevant to their roles. This prevents unauthorized actions within the system.

The system also uses JSON Web Tokens (JWT) to manage user sessions securely. Each request to protected resources is validated using the token to ensure that the user is authenticated.

Additionally, the system is designed to prevent common security issues such as unauthorized access and data exposure. Future enhancements may include advanced security features such as multi-factor authentication and secure communication protocols.

5.3 Scalability Requirements

The CampusTwin system is designed to handle growth in terms of users, data, and system functionalities. As the university environment is large and dynamic, the system should be capable of supporting an increasing number of users and activities without affecting performance.

The use of a microservices architecture allows different modules of the system to scale independently based on demand. For example, services handling incidents or notifications can be scaled separately as their usage increases.

The system is also designed to support future expansion, such as adding new features, integrating additional services, or extending the system to multiple campuses or larger environments like smart cities.

Additionally, the database and backend services should be capable of handling large volumes of data efficiently. This ensures that the system remains reliable and responsive even as the scale of operations grows.

5.4 Reliability Requirements

The CampusTwin system must provide consistent and dependable performance under normal operating conditions. It should ensure that the system remains available and functional whenever users need to access it.

The system should handle errors gracefully without causing system crashes or data loss. In case of failures, appropriate error messages should be displayed to users, and the system should recover quickly to continue normal operations.

Data integrity is also an important aspect of reliability. All user data, incident reports, and system updates should be stored accurately and consistently in the database.

The use of a microservices architecture helps in improving reliability, as failure in one service does not affect the entire system. This ensures that other parts of the system can continue functioning even if one component faces an issue.

Overall, the system should maintain stability, minimize downtime, and provide a reliable experience to all users.

5.5 Usability Requirements

The CampusTwin system is designed to be user-friendly and easy to use for all types of users, including students, staff, and administrators. The interface should be simple and intuitive so that users can easily understand and interact with the system without requiring technical knowledge.

The system should provide clear navigation, well-structured forms, and easy-to-understand visual elements such as maps and indicators. Features like incident reporting and viewing campus status should be straightforward and accessible with minimal steps.

The interface should also be responsive, allowing users to access the system from different devices such as desktops, laptops, and mobile devices. This ensures convenience and accessibility for users across the campus.

Additionally, the system should provide meaningful feedback to users, such as confirmation messages after actions like registration or incident reporting, and clear error messages when something goes wrong.

Overall, the focus is on providing a smooth and efficient user experience that encourages regular use of the system.

5.6 Maintainability Requirements

The CampusTwin system is designed to be easy to maintain and update over time. The use of a microservices architecture ensures that different parts of the system are developed and managed independently, making it easier to modify or enhance specific modules without affecting the entire system.

The code structure follows standard development practices, which helps in improving readability and simplifying debugging and testing. Proper documentation and modular design make it easier for developers to understand the system and implement changes when required.

The system is also designed to support future enhancements, such as adding new features, integrating additional services, or improving existing functionalities. This flexibility ensures that the system can evolve according to future needs.

Overall, the maintainability of the system helps in reducing development effort, minimizing downtime during updates, and ensuring long-term sustainability of the project.

6. System Design Approach

6.1 Architecture Overview (Microservices Architecture)

The CampusTwin system is designed using a **microservices architecture**, where the application is divided into multiple independent services. Each service is responsible for a specific functionality and communicates with other services through APIs.

Instead of building a single large system, the application is structured into smaller modules such as authentication, incident management, garbage monitoring and alert services. This separation makes the system more organized, scalable, and easier to maintain.

Each microservice operates independently and can be developed, deployed, and updated without affecting other parts of the system. This ensures flexibility and allows the system to handle increasing load by scaling specific services as needed.

Communication between services is handled through REST APIs, and the system is designed to support event-driven communication using message queues in future enhancements. This helps in improving performance and enabling real-time data processing.

Overall, the microservices architecture provides better modularity, scalability, and fault isolation, making the system suitable for a large and dynamic environment like a university campus.

6.2 Module Description

The CampusTwin system is divided into multiple modules, each responsible for handling a specific functionality. This modular approach ensures better organization, easier maintenance, and scalability of the system.

- **Authentication Module**
This module handles user registration, login, and security. It verifies user identity using email and password, supports OTP-based verification during registration, and manages role-based access control using JWT.
- **Incident Management Module:**
This module allows users to report incidents such as medical emergencies, security issues, and maintenance problems. It stores incident details and makes them available to the relevant authorities for action.
- **Smart Garbage Monitoring Module:**
This module tracks the status of garbage bins across the campus. It updates whether a bin is full or not and notifies maintenance staff when action is required.
- **Alert & Notification Module:**
This module generates and sends alerts to users based on events such as incidents or garbage status changes. Notifications are delivered based on user roles to ensure that the right information reaches the right users.
- **Map & Visualization Module:**
This module provides an interactive map that displays incidents and garbage bin status visually. It helps users understand campus conditions in a simple and effective way.

Overall, these modules work together to provide a complete and integrated smart campus solution.

6.3 Data Flow Overview

The CampusTwin system follows a structured flow of data between users, services, and the database to ensure smooth operation and real-time updates.

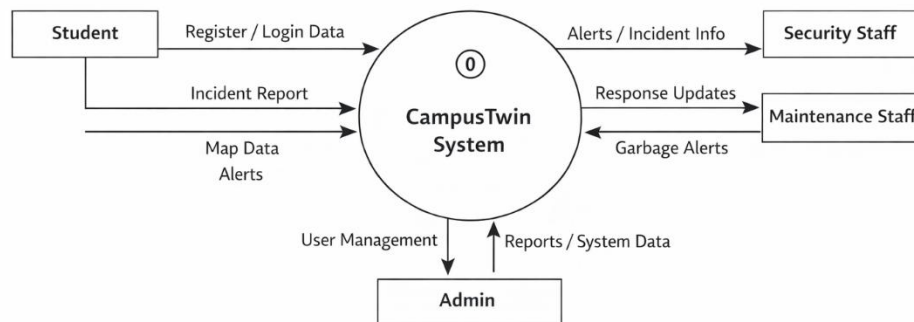
When a user interacts with the system, such as registering or reporting an incident, the request is sent from the user interface to the backend services through REST APIs. In the case of registration, user details are first verified using OTP, and only after successful verification is the data stored in the database.

For login, the system authenticates the user credentials and generates a JWT token, which is used for accessing protected resources. This ensures secure communication between the client and server. When an incident is reported, the data is stored in the database and made available to the relevant modules. Similarly, updates related to garbage bin status are processed and reflected in the system.

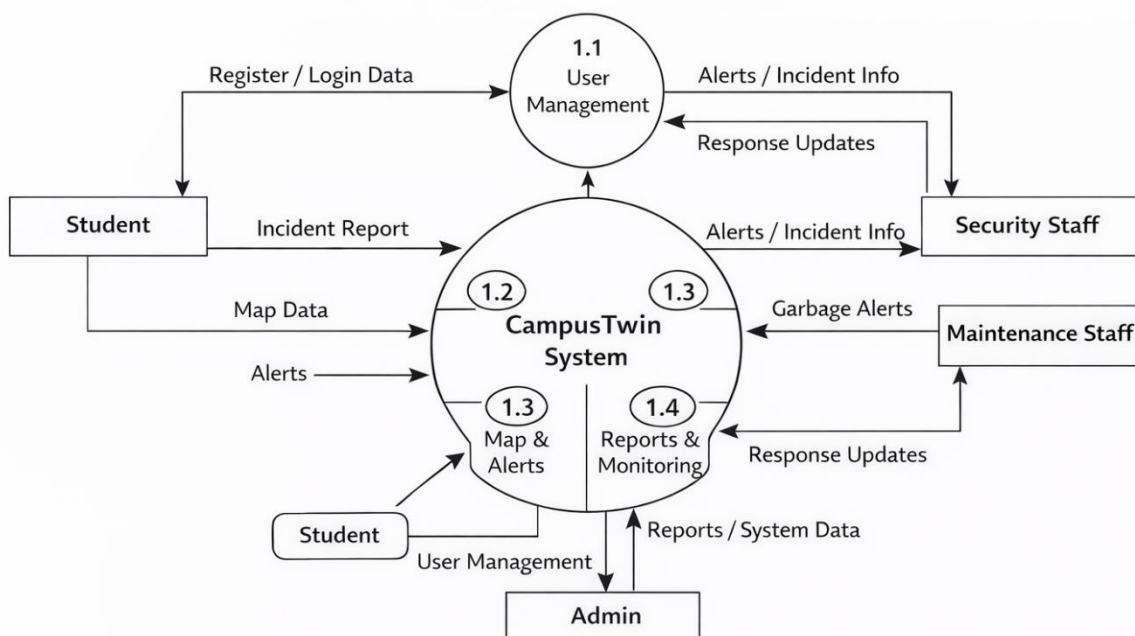
The processed data is then sent to the frontend, where it is displayed on the dashboard and map interface. This allows users and administrators to view real-time information about campus conditions.

Overall, the system ensures a continuous flow of data between users, backend services, and the database, enabling efficient monitoring and management of campus activities.

DFD Level 0: Context Diagram



DFD Level 1: CampusTwin System



6.4 Design Principles Used

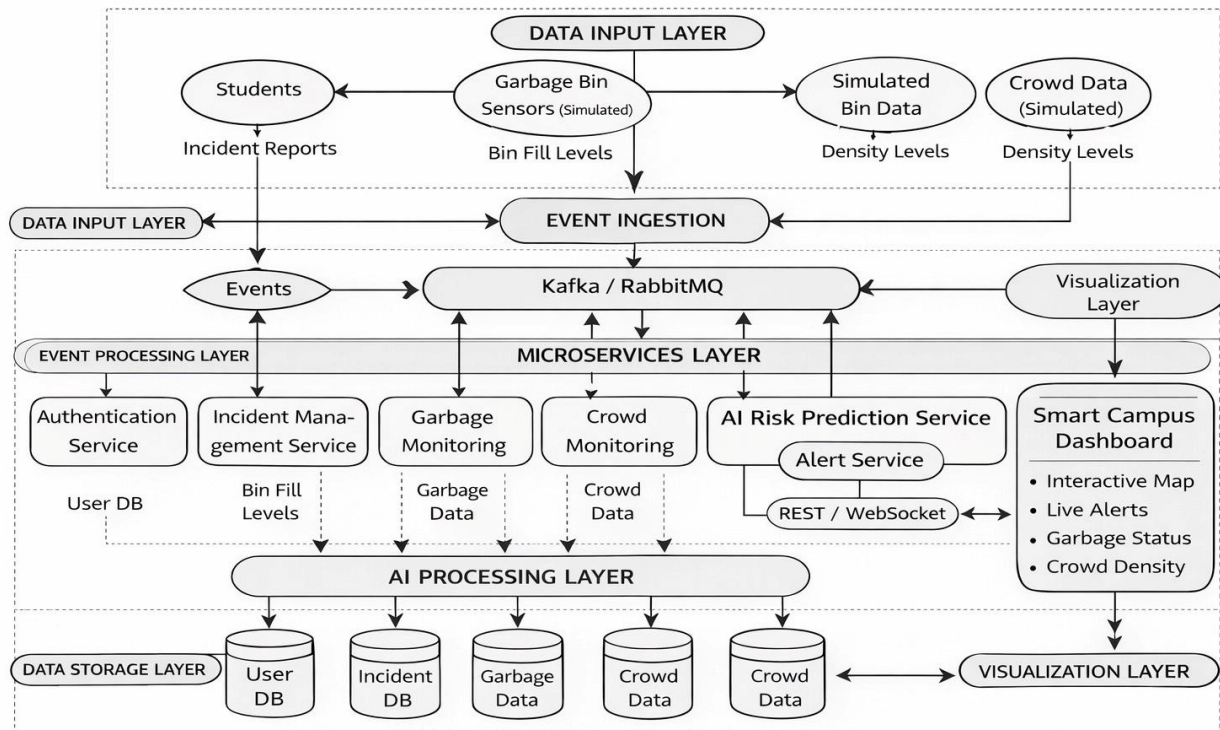
The CampusTwin system is developed by following key design principles to ensure that the application is scalable, maintainable, and easy to understand.

- **Modularity:**
The system is divided into separate modules such as authentication, incident management, and garbage monitoring. This makes the system organized and allows each module to be developed and updated independently.
- **Separation of Concerns:**
Each component of the system is responsible for a specific task. For example, authentication is handled separately from incident management, which improves clarity and reduces complexity.
- **Scalability:**
The use of microservices architecture allows the system to scale easily as the number of users or features increases. Individual services can be expanded without affecting the entire system.
- **Security First Approach:**
Security is considered from the beginning by implementing features such as OTP verification, password encryption, JWT-based authentication, and role-based access control.
- **User-Centric Design:**
The system is designed keeping the end users in mind, ensuring that the interface is simple, intuitive, and easy to use for students and staff.

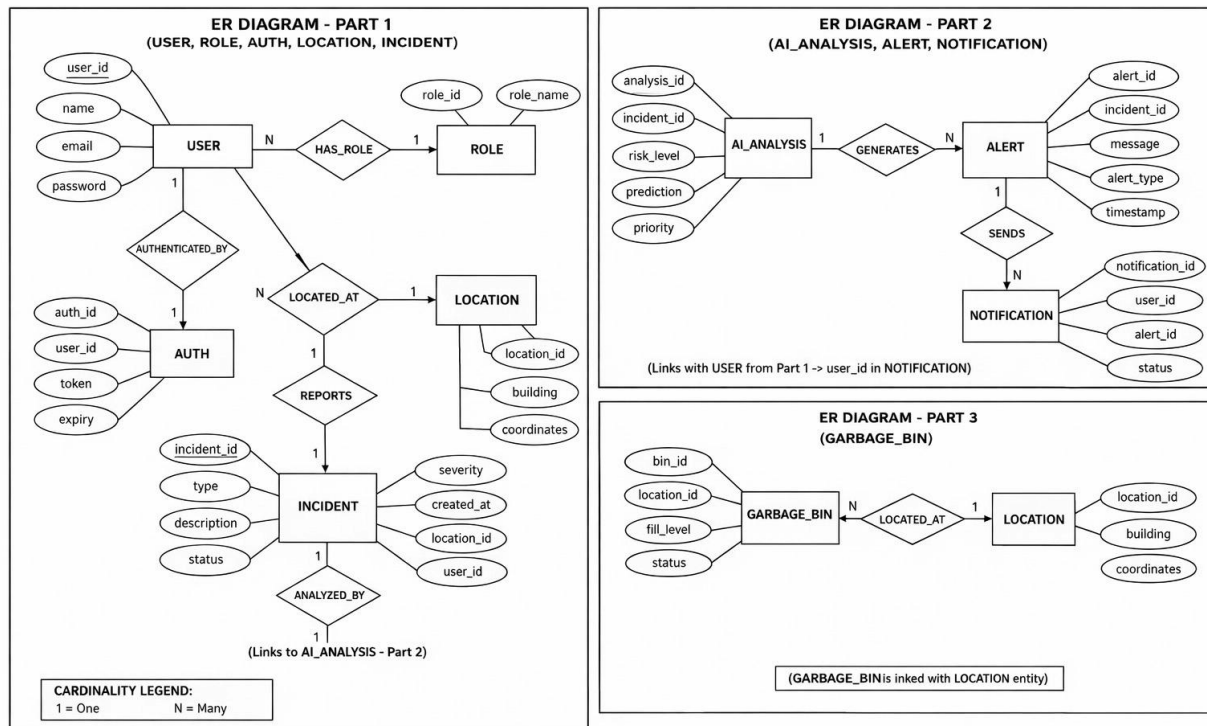
Overall, these design principles help in building a robust and efficient system that can be easily maintained and enhanced in the future.

6.5 System Architecture and ER Diagram

CAMPUSTWIN: AI DRIVEN SMART CAMPUS DIGITAL TWIN - SYSTEM ARCHITECTURE



ER DIAGRAM FOR AI DRIVEN SMART CAMPUS DIGITAL TWIN



7.1 Backend Technologies

The backend of the CampusTwin system is developed using modern technologies to ensure scalability, security, and intelligent decision-making. The system follows a microservices-based approach and integrates both Java and Python-based services.

- **Spring Boot:**
Used as the primary backend framework to build RESTful APIs and manage core functionalities such as authentication, incident management, and system coordination.
- **Spring Web:**
Used to handle HTTP requests and responses and to expose REST APIs for communication between frontend and backend services.
- **Spring Data JPA:**
Used for database interaction and object-relational mapping (ORM), enabling efficient data storage and retrieval.
- **Spring Security:**
Used to implement authentication and authorization mechanisms, ensuring secure access to the system.
- **JWT (JSON Web Token):**
Used for secure session management and stateless authentication across different services.
- **Spring Boot Validation:**
Used to validate user inputs such as registration details to maintain data consistency and integrity.
- **Lombok:**
Used to reduce boilerplate code and improve development efficiency.
- **Java Mail Sender (Spring Boot Mail):**
Used to send OTP emails during user registration for user verification.
- **Flask (Python-based AI Service):**
A separate microservice built using Flask is used to implement machine learning models. This service processes data such as incident type, location, and other inputs to predict risk levels or prioritize incidents.
- **Machine Learning Model:**
A trained model is integrated within the Flask service to analyze data and provide intelligent outputs such as severity prediction or priority classification.
- **Inter-Service Communication (REST APIs):**
The Spring Boot backend communicates with the Flask AI service through REST API calls, enabling seamless integration between the main application and the AI component.
- **Microservices Architecture:**
The system is structured into multiple independent services, including authentication, incident management, and AI services, allowing better scalability, flexibility, and maintainability.

7.2 Frontend Technologies

The frontend of the CampusTwin system is developed using modern web technologies to provide a dynamic, responsive, and user-friendly interface. It ensures smooth interaction between users and the backend services.

- **React.js:**
Used as the primary frontend library to build a component-based and interactive user interface. It allows efficient rendering and better state management for dynamic data.
- **HTML (within JSX):**
Used along with React to structure the components and define the layout of the application.
- **CSS & BootStrap:**
Used for styling the application, ensuring a clean and visually appealing design.
- **JavaScript (ES6+):**
Used for implementing application logic, handling events, and managing component behaviour.
- **Axios / Fetch API:**
Used to communicate with backend REST APIs for operations such as login, incident reporting, and data retrieval.
- **Map Integration (Leaflet / Google Maps API):**
Used to display the campus map and visualize data such as incidents and garbage bin status using markers and color indicators.
- **Responsive Design:**
Ensures that the application works efficiently across different devices including desktops, tablets, and mobile phones.

7.3 Database

The CampusTwin system uses a relational database to store and manage all application data in a structured and efficient manner.

- **PostgreSQL:**
PostgreSQL is used as the primary database for the system. It is a powerful and reliable open-source relational database that supports complex queries, data integrity, and scalability.
- **Structured Data Storage:**
The database stores important information such as user details, incident reports, garbage bin status, and other system-related data in well-defined tables.
- **Integration with Spring Data JPA:**
The backend interacts with the database using Spring Data JPA, which simplifies database operations and reduces the need for writing complex SQL queries.
- **Data Integrity and Consistency:**
Constraints such as unique fields (e.g., email and university ID) and relationships between tables ensure that the data remains accurate and consistent.

- **Scalability Support:**

PostgreSQL is capable of handling large volumes of data, making it suitable for a growing system like CampusTwin.

Overall, the database layer ensures reliable data storage, efficient retrieval, and smooth integration with backend services.

7.4 Messaging & Communication

The CampusTwin system uses different communication mechanisms to enable smooth interaction between various components and services.

- **REST APIs:**

RESTful APIs are the primary mode of communication between the frontend, backend (Spring Boot services), and the AI service built using Flask. These APIs allow different parts of the system to exchange data in a structured and efficient manner.

- **Inter-Service Communication:**

The Spring Boot backend communicates with the Flask-based AI service through HTTP requests. This enables the system to send input data (such as incident details) to the AI service and receive processed results like priority or risk level.

- **Real-Time Updates (Planned):**

The system is designed to support real-time communication mechanisms such as Server-Sent Events (SSE) or WebSockets in the future to provide live updates on the dashboard and map.

- **Event-Driven Communication (Future Scope):**

Technologies like Apache Kafka or RabbitMQ can be integrated later to enable asynchronous communication between microservices, improving scalability and performance.

Overall, the communication layer ensures efficient data exchange between system components and supports future enhancements for real-time and event-driven processing.

7.5 Tools & Frameworks

The development of the CampusTwin system involves the use of various tools and frameworks that support coding, testing, version control, and deployment.

- **Spring Boot Framework:**

Used for developing the backend services and managing application configuration.

- **React.js:**

Used for building the frontend user interface with a component-based architecture.

- **Flask Framework:**

Used to develop the AI microservice that integrates machine learning models for data analysis and prediction.

- **Postman:**

Used for testing REST APIs during development to ensure proper functionality of backend services.

- **Git & GitHub:**
Used for version control and collaboration, allowing tracking of code changes and project management.
- **Maven:**
Used for dependency management and building the Spring Boot application.
- **Visual Studio Code / IntelliJ IDEA:**
Used as development environments (IDEs) for writing and managing code.
- **Browser Developer Tools:**
Used for debugging frontend components and monitoring API calls.

Overall, these tools and frameworks help in efficient development, testing, and maintenance of the system.

8. Future Scope

In the future, the CampusTwin system can be further enhanced by implementing several advanced features and improvements.

The system will be extended to integrate real-time data using IoT devices such as smart garbage sensors and surveillance systems. This will replace simulated data with actual live inputs, making the system more accurate and reliable.

The AI module will be improved by developing and training more advanced machine learning models to provide better predictions, such as incident severity classification and intelligent alert prioritization.

A dedicated mobile application will be developed to allow users to report incidents and receive notifications more conveniently. This will improve accessibility and user engagement.

The system will also be enhanced by implementing real-time communication using technologies like WebSockets or event-driven messaging systems such as Kafka to provide live updates across the platform.

Finally, the system can be scaled to support multiple campuses or extended to a smart city-level solution, making it applicable beyond a single university environment.

9. Appendix

The appendix section includes additional information that supports the understanding and development of the CampusTwin system.

It may contain supporting materials such as sample API request and response formats, database structure details, and any assumptions made during the design of the system. This section can also include diagrams such as Data Flow Diagrams (DFD), Entity Relationship (ER) diagrams, or architecture diagrams that provide a visual representation of the system.

Any additional technical details, configurations, or references that are not included in the main sections of the document can also be placed here for further clarification.

Overall, the appendix serves as a supplementary section that provides extra information to support the implementation and understanding of the system.

